Wingspan
Prospectus

Carter Feng, Fedhil Al-Shammary, Ansh Jain, Tengyi (Jake) Guo
October 2025



Overview

1. Outline

i. One Page Outline.....1

2. Modeling Design

i. UML.....2
ii. Explanation of UML.....3
iii. Game User Interface.....9
iv. Explanation of GUI.....14
v. Round of Play.....16

3. Implementation

i. Algorithms.....18
ii. Testing Logic.....21
iii. Testing Graphics.....25
iv. Potential Conflicts.....29

4. Timeline

i. General Timeline.....30
ii. Division of Workload.....37

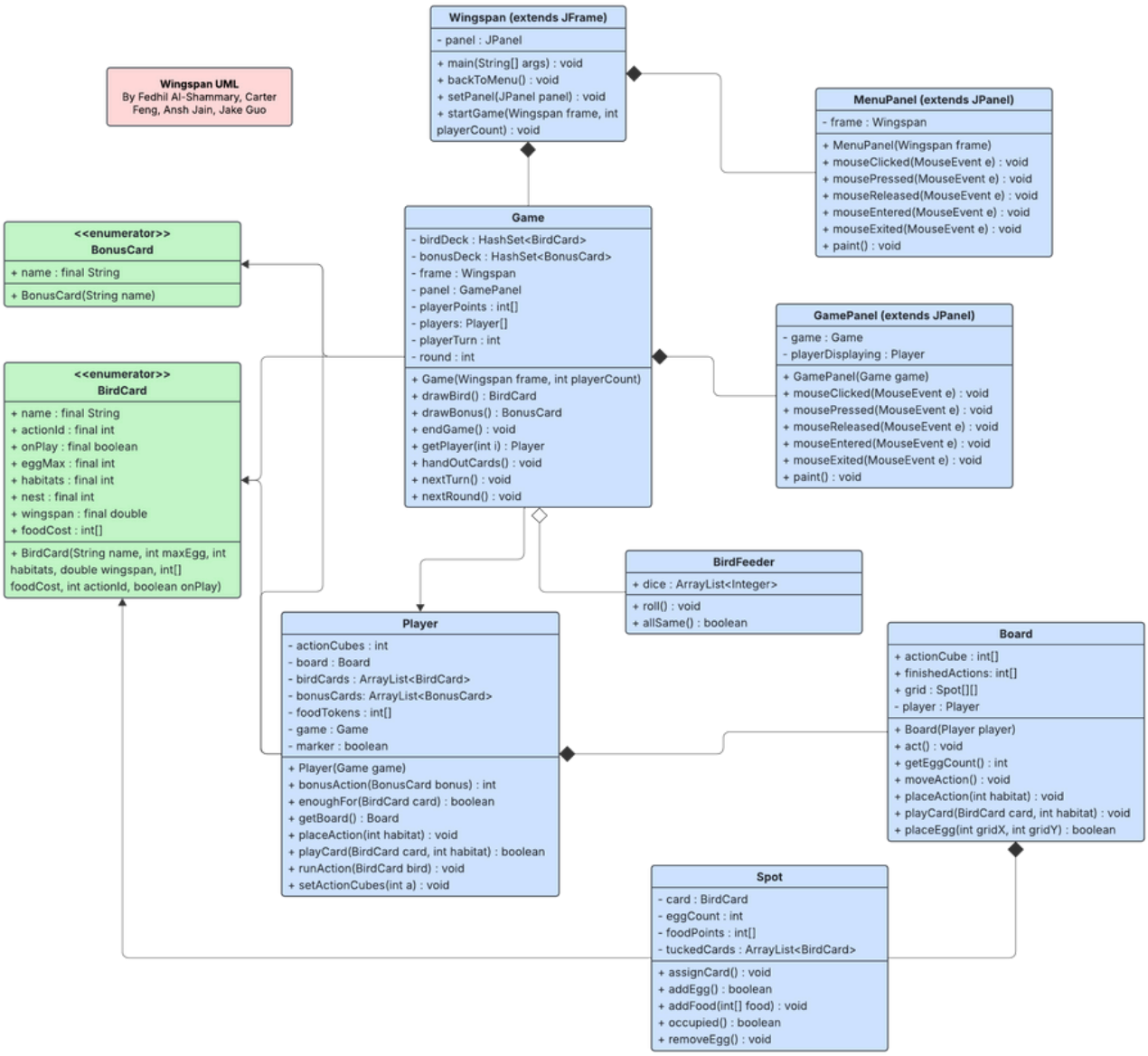
Outline

We will be programming the game “Wingspan” in Java. Wingspan is an engine-building board game played with up to 5 players, where the main objective is for players to try to collect bird cards and gain points over the span of 4 rounds, depending on a point index randomized at the beginning of the game.
This prospectus will outline our progress and plan for creating this game, providing insight on how Team Jake will divide up work, execute the numerous behaviors and functions of the board game, and serve as a reference for future improvements or changes to the game.

Our Project:

- Our project will be played between two to five players, with each owning their individual board and randomized birds.
- The game will be divided into a total of 7 classes and 2 enumerators: the Wingspan class, the MenuPanel class, the GamePanel class, the Game class, the Player class, the Board class, the Spot class, the BirdCard enum, and the BonusCard enum. The game will mainly run through the Game class, while the graphics will be handled by the MenuPanel and GamePanel classes.
- This prospectus contains:
  - **UML:** the barebones of the program and rough outline as to what the plan is going forward into programming the game
  - **Explanation of UML:** An in depth guide into explaining each method and functionality of the UML diagram
  - **GUI:** A basic draft of how the game will look graphically
  - **Explanation of GUI:** An in depth guide into explaining how we plan to draw out the game
  - **Round of Play:** A walkthrough of the available options the players can do during rounds
  - **Algorithms:** A walkthrough of the steps in the game that will happen during rounds/gameplay
  - **Testing Logic:** Possible conflicts and problems that may occur with the logic of the game
  - **Test Graphics:** Possible conflicts and problems that may occur with the graphics of the game
  - **Timeline:** How we plan on completing the game on time

UML


Wingspan UML
By Fedhil Al-Shammary, Carter Feng, Ansh Jain, Jake Guo
Wingspan (extends JFrame)
- panel : JPanel
+ main(String[] args) : void
+ backToMenu() : void
+ setPanel(JPanel panel) : void
+ startGame(Wingspan frame, int playerCount) : void
MenuPanel (extends JPanel)
- frame : Wingspan
+ MenuPanel(Wingspan frame)
+ mouseClicked(MouseEvent e) : void
+ mousePressed(MouseEvent e) : void
+ mouseReleased(MouseEvent e) : void
+ mouseEntered(MouseEvent e) : void
+ mouseExited(MouseEvent e) : void
+ paint() : void
<<enumerator>>
BonusCard
+ name : final String
+ BonusCard(String name)
Game
- birdDeck : HashSet<BirdCard>
- bonusDeck : HashSet<BonusCard>
- frame : Wingspan
- panel : GamePanel
- playerPoints : int[]
- players: Player[]
- playerTurn : int
- round : int
+ Game(Wingspan frame, int playerCount)
+ drawBird() : BirdCard
+ drawBonus() : BonusCard
+ endGame() : void
+ getPlayer(int i) : Player
+ handOutCards() : void
+ nextTurn() : void
+ nextRound() : void
GamePanel (extends JPanel)
- game : Game
- playerDisplaying : Player
+ GamePanel(Game game)
+ mouseClicked(MouseEvent e) : void
+ mousePressed(MouseEvent e) : void
+ mouseReleased(MouseEvent e) : void
+ mouseEntered(MouseEvent e) : void
+ mouseExited(MouseEvent e) : void
+ paint() : void
<<enumerator>>
BirdCard
+ name : final String
+ actionId : final int
+ onPlay : final boolean
+ eggMax : final int
+ habitats : final int
+ nest : final int
+ wingspan : final double
+ foodCost : int[]
+ BirdCard(String name, int maxEgg, int habitats, double wingspan, int[] foodCost, int actionId, boolean onPlay)
BirdFeeder
+ dice : ArrayList<Integer>
+ roll() : void
+ allSame() : boolean
Player
- actionCubes : int
- board : Board
- birdCards : ArrayList<BirdCard>
- bonusCards: ArrayList<BonusCard>
- foodTokens : int[]
- game : Game
- marker : boolean
+ Player(Game game)
+ bonusAction(BonusCard bonus) : int
+ enoughFor(BirdCard card) : boolean
+ getBoard() : Board
+ placeAction(int habitat) : void
+ playCard(BirdCard card, int habitat) : boolean
+ runAction(BirdCard bird) : void
+ setActionCubes(int a) : void
Board
+ actionCube : int[]
+ finishedActions: int[]
+ grid : Spot[][]
- player : Player
+ Board(Player player)
+ act() : void
+ getEggCount() : int
+ moveAction() : void
+ placeAction(int habitat) : void
+ playCard(BirdCard card, int habitat) : void
+ placeEgg(int gridX, int gridY) : boolean
Spot
- card : BirdCard
- eggCount : int
- foodPoints : int[]
- tuckedCards : ArrayList<BirdCard>
+ assignCard() : void
+ addEgg() : boolean
+ addFood(int[] food) : void
+ occupied() : boolean
+ removeEgg() : void

UML Explanation

The Class Wingspan (Extends JFrame)

Overview: This class is the runner class of the program and contains methods required for transitioning between and displaying panels.

Attributes

- **JPanel panel:** The current panel being displayed on the frame

Methods

- **void main(String[] args):** Program runner function; first displays the MenuPanel to the player
- **void backToMenu():** Returns the player to the menu after the game is over
- **void setPanel(JPanel panel):** Set the attribute panel to the provided panel and update which panel is being displayed
- **void startGame(Wingspan frame, int playerCount):** Creates a new Game and runs setPanel to update the panel to a new GamePanel

The Class Game

Overview: This class is the bulk of the program and contains attributes and methods required for initialization, scoring, and turn mechanics. It also contains global variables required throughout the entire game.

Attributes

- **HashSet<BirdCard> birdDeck:** A set of all the BirdCards in the game, one created for each instance of the enum BirdCard
- **HashSet<BonusCard> bonusDeck:** A set of all the BonusCards in the game, one created for each instance of the enum BonusCard
- **Wingspan frame:** The instance that contains necessary methods for transition of panels and the beginning and ending of the game
- **GamePanel panel:** The GamePanel used to communicate with the graphical user interface
- **int[] playerPoints:** An array containing each player's points
- **Player[] players:** An array containing all Player objects used to play the game, one for each player
- **int playerTurn:** The index of the Player instance corresponding to the player whose turn it is
- **int round:** The current round being played

Methods

- **Game(Wingspan frame, int playerCount):** Initializes the object to create the players array with playerCount amount of Players and to hold the parent class in the frame attribute
- **BirdCard drawBird():** Removes and returns a BirdCard from the HashSet birdDeck
- **BonusCard drawBonus():** Removes and returns a BonusCard from the HashSet bonusDeck
- **void endGame():** Calculates scores and displays the leaderboard/victors; ends the game
- **Player getPlayer(int i):** Returns the Player at index i of the array players
- **void handOutCards():** Draws five BirdCards per player allows the player to select a total of five items out of five foodTokens and the five cards, then allows the player to select one of two drawn BonusCards; discards unused cards
- **void nextTurn():** Sets the playerTurn to the next index and runs all necessary functions for the player's turn
- **void nextRound():** Sets up the next round by decrementing each Player's starting actionCubes and adding up points based on the criteria of the current round

The Class MenuPanel (Extends JPanel)

Overview: This class is the graphical user interface for the startup menu and communicates primarily the number of players to the Wingspan class.

Attributes

- **Wingspan frame:** The Wingspan instance containing methods for panel transition and game startup

Methods

- **MenuPanel(Wingspan frame):** Initializes the object to hold the parent class in the frame attribute
- **void mouseClicked(MouseEvent e):** Communicates the information of the graphical user interface to the Wingspan class, primarily the number of players and when to start the round
- **void mousePressed(MouseEvent e):** Necessary for the MouseListener interface; unused
- **void mouseReleased(MouseEvent e):** Necessary for the MouseListener interface; unused
- **void mouseEntered(MouseEvent e):** Necessary for the MouseListener interface; unused

- **void mouseExited(MouseEvent e):** Necessary for the MouseListener interface; unused
- **void paint():** Updates graphics in response to the user's clicks

The Class GamePanel (Extends JPanel)

Overview: This class is the graphical user interface for the main game and comunicates actions launched by the player. It will display the board of each player and will give the option to display other players' boards during each player's turn.

Attributes

- **Game game:** The Game instance that the graphical user interface communicates with
- **Player playerDisplaying:** The Board of the Player being displayed

Methods

- **GamePanel(Wingspan frame):** Initializes the object to hold the parent class in the frame attribute and to set the playerDisplaying attribute
- **void mouseClicked(MouseEvent e):** Communicates the information of the graphical user interface to the Game class including the action being done and the board being displayed
- **void mousePressed(MouseEvent e):** Necessary for the MouseListener interface; unused
- **void mouseReleased(MouseEvent e):** Necessary for the MouseListener interface; unused
- **void mouseEntered(MouseEvent e):** Necessary for the MouseListener interface; unused
- **void mouseExited(MouseEvent e):** Necessary for the MouseListener interface; unused
- **void paint():** Updates graphics in response to the user's clicks

The Class Player

Overview: This class will contain attributes related to each player's items and board as well as functions required for playing cards and executing actions.

Attributes

- **int actionCubes:** The remaining number of actions the corresponding player can take in a given round
- **Board board:** The Board class corresponding to this Player instance
- **ArrayList<BirdCard> birdCards:** The hand of BirdCards corresponding to the player
- **ArrayList<BonusCard> bonusCards:** The hand of BonusCards corresponding to the player

- **int[] foodTokens:** An array of size 4 holding the corresponding player's number of food tokens of each category
- **Game game:** The Game instance that the Player communicates with for drawing cards (establishes a bidirectional relationship)
- **boolean marker:** Whether or not the corresponding player holds the first player marker

Methods

- **Player(Game game):** Initializes the Player object with a bidirectional relationship to the Game instance
- **int bonusAction(BonusCard bonus):** Returns the bonus points given by the bonus
- **boolean enoughFor(BirdCard bird):** Determines whether or not the player has enough food tokens and eggs to play the given BirdCard
- **Board getBoard():** Returns the Board instance for Game and GamePanel
- **void placeAction(int habitat):** Executes the action for the given habitat before decrementing the actionCube's column to execute preceeding actions
- **boolean playCard(BirdCard, int habitat):** Plays the given BirdCard in the given habitat if requirements are met; returns whether or not the play was legal and successful
- **void runAction(int id):** Runs the action corresponding to the given actionId
- **void setActionCubes(int a):** Sets the actionCubes by the Game class

The Class Board

Overview: This class will be used primarily for storing a grid of the cards played by each player as well as food points and eggs stored on the board. It will contain necessary functions for modifying such attributes as well as for moving and launching actions.

Attributes

- **int[] actionCube:** The x- and y- values of the current actionCube on the Board
- **int[] finishedActions:** The y-value of finished actionCubes on the board for the purpose of displaying
- **Spot[][] grid:** The matrix of Spots that hold BirdCards, eggs, and food tokens
- **Player player:** The Player instance corresponding to the Board

Methods

- **Board(Player player):** Initializes the Board with a bilateral relationship to the Player

- **void act(): Runs** the action corresponding to the Spot on the coordinate of the actionCube
- **int getEggCount():** Returns the sum of the eggs on all the Spots in the grid
- **void moveAction():** Moves the actionCube to the left and continues to act until the actionCube is finished, where it sets the coordinate values of actionCube to -1 and increments the corresponding value in finishedActions
- **void placeAction(int habitat):** Decrements the Player's actionCubes and sets the coordinate of the Board's actionCube
- **void playCard(BirdCard card, int habitat):** Places the BirdCard in the corresponding Spot in the grid and gives the option to run the Action onPlay
- **boolean placeEgg(int gridX, int gridY):** Places the egg on the Spot corresponding to the given coordinates; returns whether or not the placement was successful

The Class BirdFeeder

Overview: This class is simply meant to represent the bird feeder and the management of the dice.

Attributes

- **ArrayList<Integer> dice:** The five dice the BirdFeeder contains

Methods

- **void roll():** Randomly assigns five values from 1-6 to the dice
- **boolean allSame():** Returns whether or not all the values are the same in order to prompt the user to reroll

The Class Spot

Overview: This class will be representative of each spot on the player's board. It will contain attributes of each spot - such as the card holding the spot, food points, and eggs - as well as functions for modifying attributes (i.e. placing a card, adding and removing eggs and food points).

Attributes

- **BirdCard card:** The BirdCard occupying the Spot, if any
- **int eggCount:** The number of eggs on the Spot, if occupied
- **int[] foodPoints:** The number of each food token cached as a point on the Spot, if occupied
- **ArrayList<BirdCard> tuckedCards:** The BirdCards tucked underneath the occupying BirdCard, if occupied

Methods

- **void assignCard():** Assigns a BirdCard to occupy the Spot if unoccupied
- **boolean addEgg():** Adds an egg to the Spot if possible; returns whether or not the placement was successful
- **void addFood(int[] food):** Adds a food point to the Spot
- **boolean occupied():** Returns whether or not the Spot has a BirdCard occupying it
- **void removeEgg():** Decrements the eggCount of the Spot

The Enum BirdCard

Overview: This enum structures the bird cards with necessary values for the initialization of all of them in one long file. It contains constants that will vary for each card but should otherwise not be modified.

Attributes

- **final String name: The name of the BirdCard**
- **final int actionId: The ID of the action that the BirdCard holds**
- **final boolean onPlay:** Whether the action is upon the BirdCard's placement or upon activation
- **final int eggMax:** The number of eggs that can be placed on the Spot the BirdCard occupies
- **final int habitats:** A bitmask of the habitats the BirdCard can be placed in
- **final double wingspan:** The wingspan of the BirdCard
- **int[] foodCost:** The number of food tokens of each variety required to play the BirdCard

Methods

- **BirdCard(String name, int maxEgg, int habitats, double wingspan, int[] foodCost, int actionId, boolean onPlay):** Initializes the BirdCard with respective values

The Enum BonusCard

Overview: This enum is primarily for the sake of encapsulation so as to correspond with images and actions easier.

Attributes

- **final String name:** The name of the BonusCard

Methods

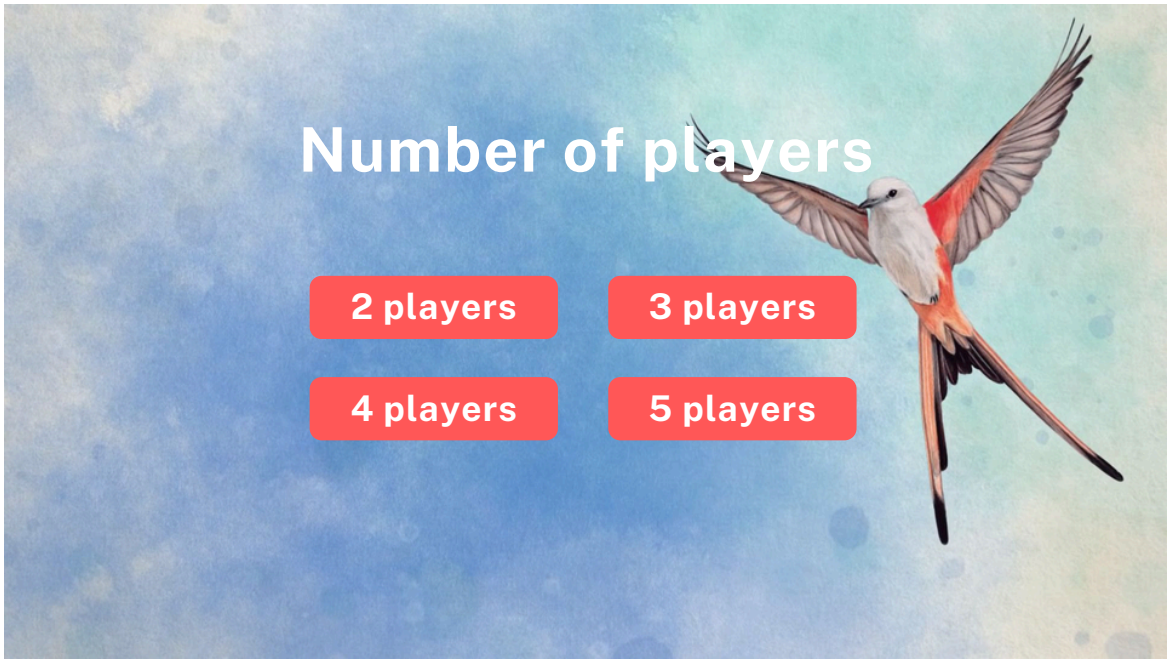
- **BonusCard(String name)**

GUI

Title Screen

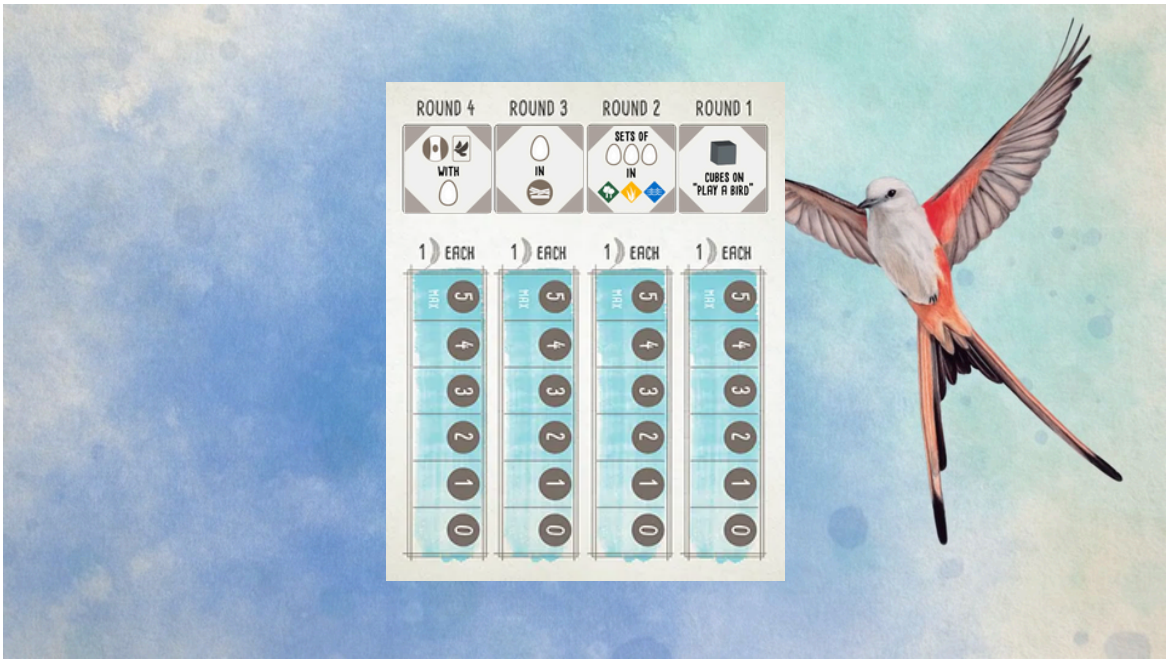


Setup Screen

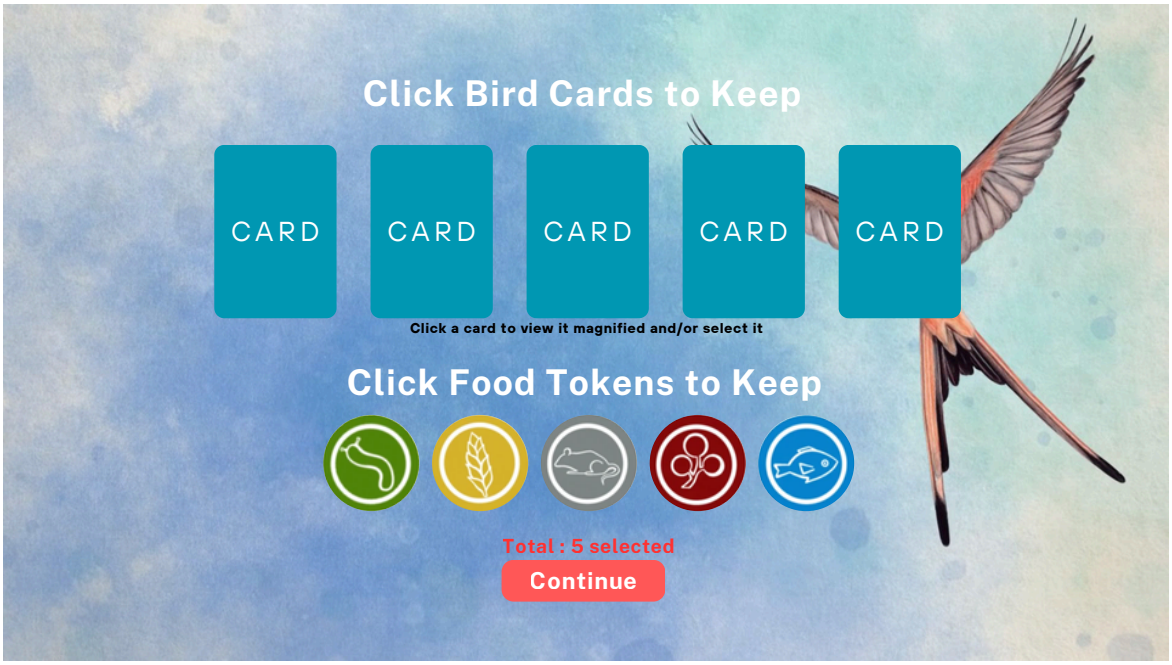


GUI

Round Scoring

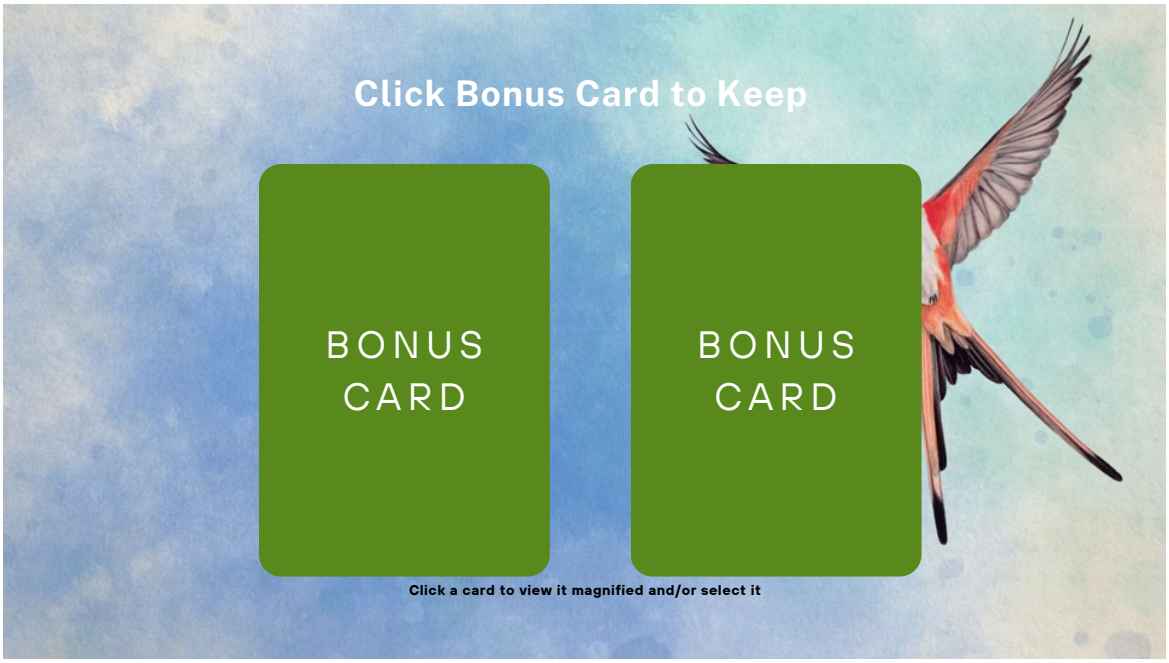


Selection Screen at the beginning

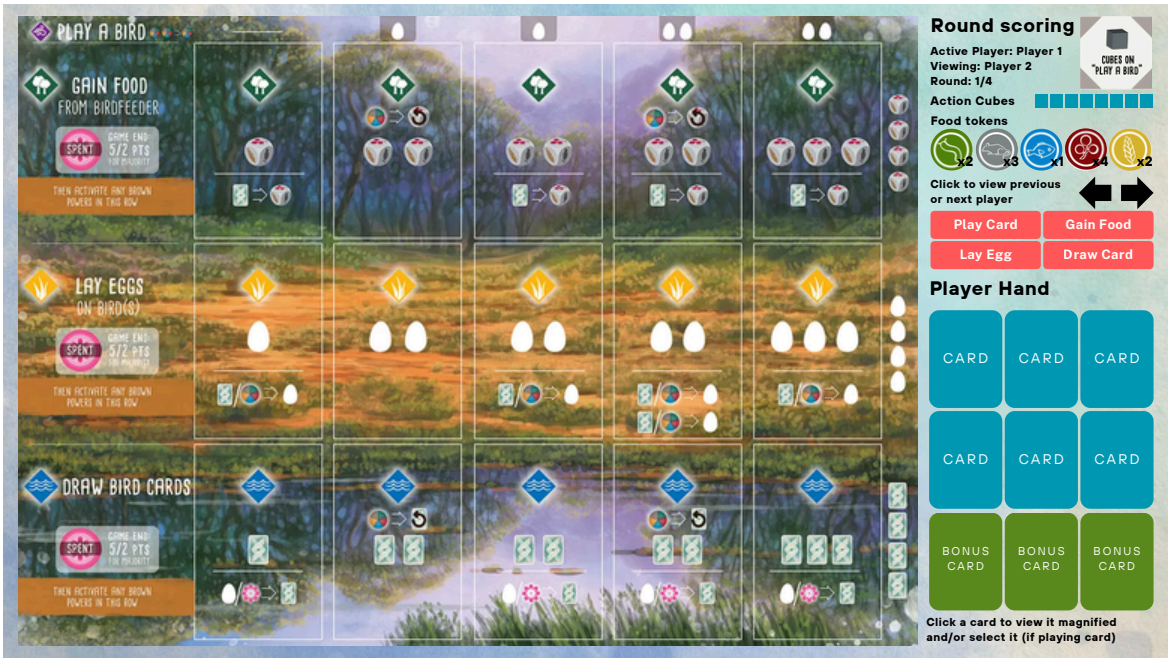


GUI

Selection Screen at the beginning

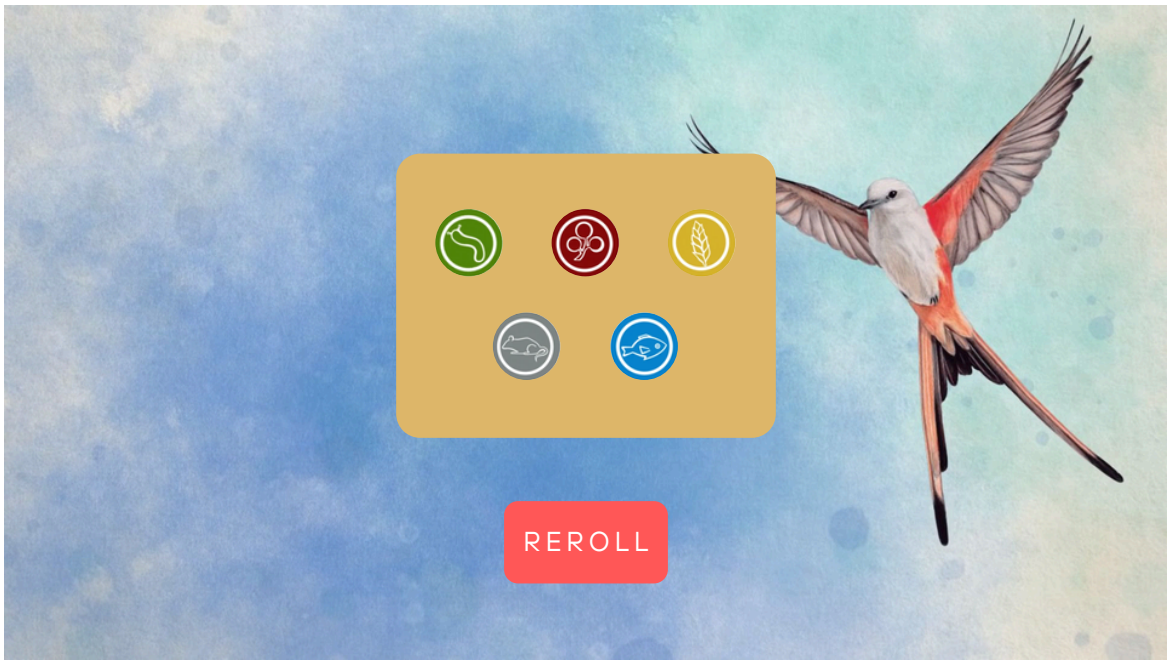


Game Screen

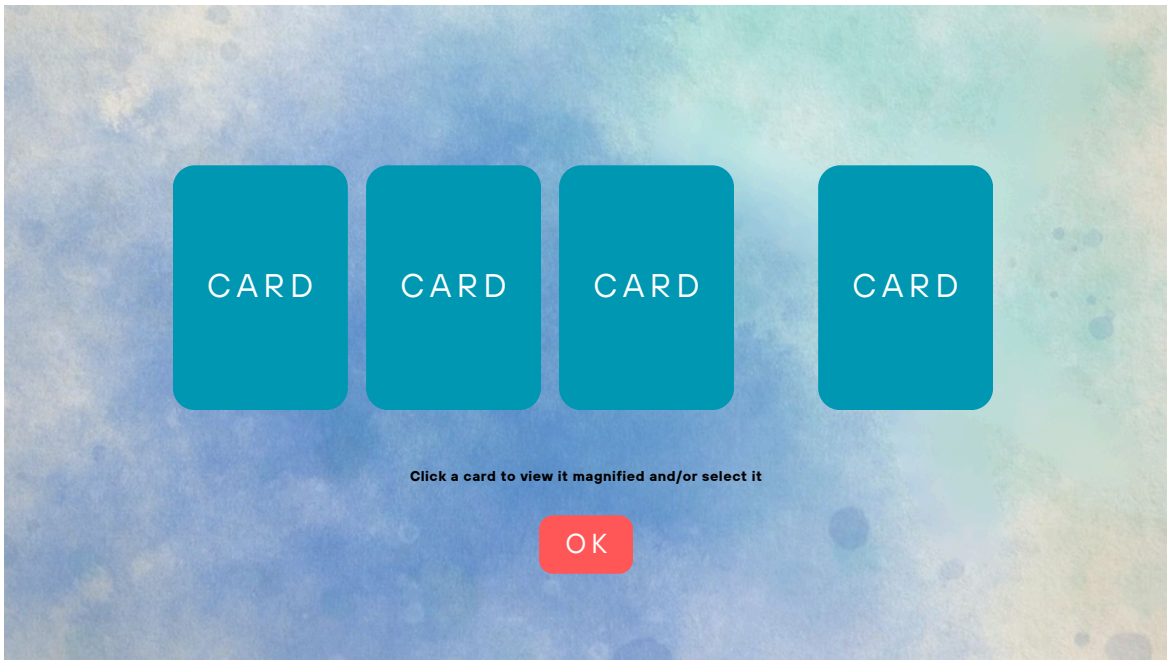


GUI

Bird Feeder Screen

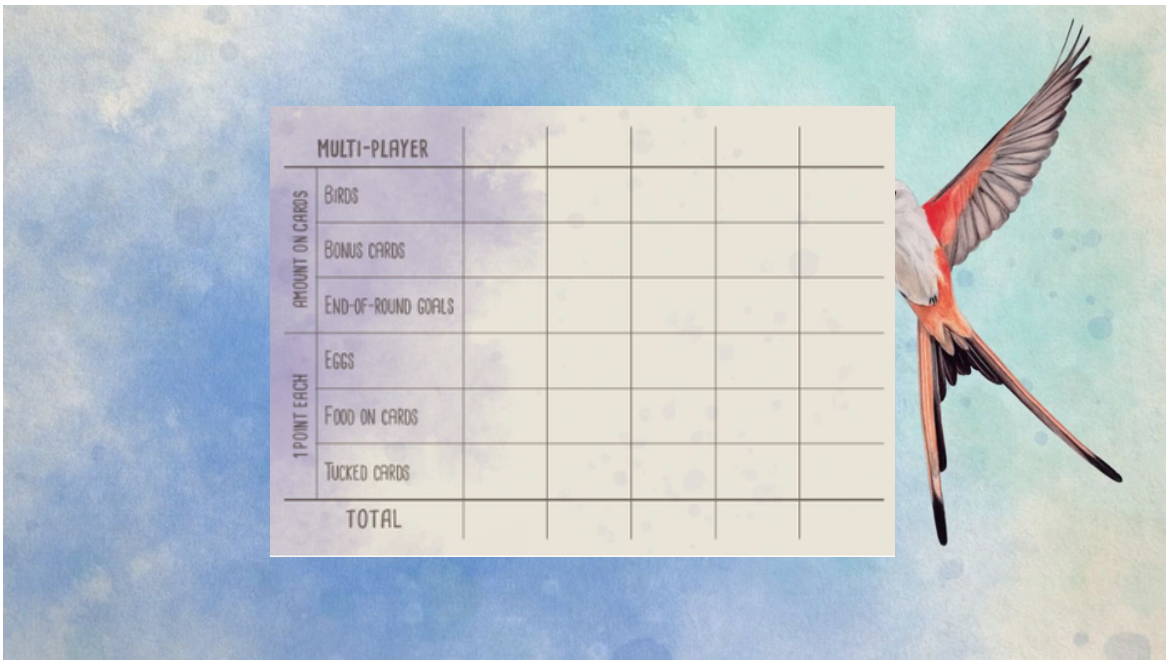


Selecting Cards (Deck/Tray)



GUI

Scoring at the end of the rounds and game



End Game Screen



GUI Explanation

Title Screen/Menu
After the JAR file is run, the menu screen/a picture of Wingspan will be displayed. The user will be able to click a button to start the game.

Setup Menu
We will adding a basic player selection interface of four buttons for 2-5 players. After clicking a button, the game will start with 2-5 players based on the button the user has clicked.

Round Scoring
After the user has clicked the button to start the game, the first screen that will pop up will be a round scoring indicator. This will tell the users what the scoring per round will be based on, and will also pop up between rounds in order to show the current scores and also to remind players of the next scoring indicator.

Selection Screen
Each player will choose which bird cards to keep and which food tokens to keep, and if the # of bird cards and # of tokens do not add to 5, they will not be able to continue (button won't be able to be clicked) They will be able to click a bird card in order to view it in a magnified view.

Selection Screen (bonus cards)
When the player needs to choose a bonus card from the two given options, this screen will appear to display the choices. They will be able to click a card in order to view it in a magnified view, and if they do not wish to select it, they can click the "cancel" button.

Game Screen
The game screen will show the player's board, active player, round, available action cubes, and scoring. The player will be able to click arrows in order to view other players boards.

Bird Feeder Screen

When a user needs to take a food token, a screen will appear showing the die in the bird feeder. The user can then click a food token to take it, and if all foods within the feeder are the same, they will be able to reroll all dice.

Selection Screen (cards)

If the player needs to choose a card from the tray/deck, this screen will pop up. They will be able to click a card in order to view it in a magnified view, and if they do not wish to select it, they can click the “cancel” button. (this will not happen for the deck pile).

Scoring at the end of the rounds and the game

This will display the points gained after rounds and the game.

End Game Screen

This will display who won the game.

Round of Play

Player Selection

- The players will select “Click to Play” and be redirected to the player selection screen, where they are able to select the number of players participating ranging from 2-5.
- The players will then be shown the round-scoring criteria.
- Each player will be made to choose between five food tokens and five bird cards for a total of five items. They will then be made to choose between two bonus cards before the next player makes their selections. Unused cards are discarded.

Actions and View

- Once all players have made their selections, the first player’s board is displayed so that they may make their turn. Each player’s board displays information regarding cards in the player’s hand; other players are expected not to look.
- The player can select which other player’s board to display in order to view public information such as played cards and food tokens. The scoring criterium for the round is listed to the side as well as the action cubes the player has remaining for the round.
- Each player is given one turn for each action cube. The player is given four options to choose from:

1. Playing a Card

- The player selects a card from their hand that they wish to play.
- If the player does not have enough food tokens or eggs to play the card, the screen will display such. If the player does have enough, they will select food tokens to exchange for the card’s placement, then eggs corresponding to the column once a habitat has been selected.
    - After the card is played, the player has the option to activate the card’s “On Play” ability.
- The action cube is then moved to the finished section to the left of the “Play a Bird” label.

2. Gaining Food

- The player places their action cube on the left-most exposed space on the “Gain Food” row.
- The player selects from the bird feeder the food tokens that they would like to claim corresponding to the amount on the space, removing each die selected. If all dice are on the same side before selection, the player has the option to reroll.

3. Laying Eggs

- The player places their action cube on the left-most exposed space on the “Lay Eggs” roll.
- The player is given the corresponding amount of eggs and is required to place them on any bird cards that can hold the eggs.

4. Drawing a Bird Card

- The player places their action cube on the left-most exposed space on the “Draw Bird Cards” row.
- The player draws the corresponding amount of bird cards into their hand.

Bird Activation

- If the action was any of the three above, the action cube moves one down the row and gives the player the option to activate each card crossed over. After all cards have been crossed, the action cube is moved to the finished section of the corresponding row.
- Once in between turns, players who have cards that activate in-between turns are prompted to do so if desired.

Round Scoring

- The round continues until all players have used all action cubes. After the round is over, the players are awarded points based on the criterium for the current round.
- The next round is started with each player having one less action cube and a new criteria.
- Four rounds are played before the game is over.

Final Scoring

- After all four rounds have been played and scored, the final score is calculated by adding the additional bonus points given from the cached food items as well as the bonus cards.
- The leaderboard is displayed with the results of the game ordered by ranking.

Algorithms

Setup Algorithm

- The main method in the Wingspan class starts the game on a newly created MenuPanel.
- The MenuPanel contains a basic interface to select the number of players in the game and a play button. It runs the startGame method in the Wingspan class afterwards with the proper variables and a newly initialized GamePanel, setting the GamePanel as the panel to be displayed by the Wingspan class.
- The startGame method creates the Game class, which instantiates new Player classes (that instantiate their own Board classes with grids of Spot instances), creates the global decks, and draws five cards; two bonus cards; and five food tokens for each player to select a total of five items and one bonus card. Others are discarded.
- The Round Algorithm is run.
- The endGame method of the Game class will run the bonusAction method for each card and add the bonus points to each player in the playerPoint array.
- The leaderboard and a victor will be displayed by the GamePanel as well as an option to restart. Upon restarting, the Setup Algorithm runs again.

Round Algorithm

- The Game class sets the first player's (or the next player from the previous round's) marker to true and displays their board on the GamePanel.
- The Turn Algorithm is run until the nextRound method is called. The Game class increments the round attribute and runs the Scoring Algorithm.
  - If the round attribute of the Game reaches 4, the endGame method is called in the Game class, and the Round Algorithm terminates.
  - Otherwise, the Round Algorithm continues to loop.
- Each Player's actionCubes attribute is initialized to one less than the round before.the GamePanel with the option to restart. Restarting will display the MenuPanel once again to restart the overall game loop.

Turn Algorithm

- The GamePanel listens for a mouseClicked event and internally manages code relating to graphics (which may include basic variables not detailed in the UML) until an action is performed.
- The Player's actionCubes will be decremented upon any action selected by a player, and the Board's actionCube will have its coordinate set to the respective spot.
  - If the action is playing a card, the Card Placement Algorithm is run.
  - Otherwise, the Action Algorithm is run.
- After the action is completed, the nextTurn method of the Game class will be run, restarting the Turn Algorithm on the next player's turn. If the next Player's actionCubes count is equal to 0, the nextRound method is called, and the Turn Algorithm terminates.

Card Placement Algorithm

- The playCard method is run in the respective Player class to determine if the card was legal to play.
- If legal, the method plays it by calling the subsequent playCard method in its Board instance, returning true if successful and false otherwise.
- If successful, the GamePanel will display the option to run the card's on-play effect, running the runAction method in the Player class if so.
- The respective finishedActions count in the Board instance will be incremented and the actionCube will be set to [-1, -1].

Action Algorithm

- The placeAction method will be run to launch a loop in the Board instance where the initial position of the actionCube, an unoccupied spot, will affect the player with its respective trait, be that adding eggs, gaining food, or drawing a bird card.
- The GamePanel will give the option to trade if the spot on the board offers, running the respective trade.
- The loop will continued with the moveAction method - the method to move the actionCube left - being called until the actionCube's first value is -1, at which point the actionCube will be set to [-1, -1] and the respective finishedActions count will be incremented.
  - The GamePanel will give the option to activate each card it crosses over if possible. If activated, the runAction method will be run in the Player instance to activate the action of the BirdCard's actionId.

Turn Algorithm

- The GamePanel listens for a mouseClicked event and internally manages code relating to graphics (which may include basic variables not detailed in the UML) until an action is performed.

- The Player's actionCubes will be decremented upon any action selected by a player, and the Board's actionCube will have its coordinate set to the respective spot.
  - If the action is playing a card, the Card Placement Algorithm is run.
  - Otherwise, the Action Algorithm is run.
- After the action is completed, the nextTurn method of the Game class will be run, restarting the Turn Algorithm on the next player's turn. If the next Player's actionCubes count is equal to 0, the nextRound method is called, and the Turn Algorithm terminates.

Card Placement Algorithm

- The playCard method is run in the respective Player class to determine if the card was legal to play.
- If legal, the method plays it by calling the subsequent playCard method in its Board instance, returning true if successful and false otherwise.
- If successful, the GamePanel will display the option to run the card's on-play effect, running the runAction method in the Player class if so.
- The respective finishedActions count in the Board instance will be incremented and the actionCube will be set to [-1, -1].

Action Algorithm

- The placeAction method will be run to launch a loop in the Board instance where the initial position of the actionCube, an unoccupied spot, will affect the player with its respective trait, be that adding eggs, gaining food, or drawing a bird card.
- The GamePanel will give the option to trade if the spot on the board offers, running the respective trade.
- The loop will continued with the moveAction method - the method to move the actionCube left - being called until the actionCube's first value is -1, at which point the actionCube will be set to [-1, -1] and the respective finishedActions count will be incremented.
  - The GamePanel will give the option to activate each card it crosses over if possible. If activated, the runAction method will be run in the Player instance to activate the action of the BirdCard's actionId.

Scoring Algorithm

- The Game class will evaluate the round attribute and provide a specific criteria on which to assign points.
- A nested loop runs through each Spot of each Board of each Player instance and adds points to the playerPoints array based on the given criteria.

Testing Logic

- After the user starts the game, we will need to guarantee that the number of players detected is correctly instantiated and the parameter properly carries over the value. It is also expected that moving through the player boards, it will iterate through the right number of players. This can be verified by using print statements and through manually testing.
- For each player, we will need to verify that each player is only able to get 5 items (bird cards, food tokens) total and that each card not chosen is discarded. We will also need to check and make sure that the player can only select one bonus card at the beginning of the game, and the other card is discarded. This can be done via print statements.
- We will need to verify that each player has access to one of four actions (play card, draw card, lay egg, or gain food) and that all actions can be performed legally. We will make use of print statements to display each player's stats to ensure accuracy in variable management.
- When playing a card down, we will need to make sure that the game will pop up either an error message or not allow the player to do illegal actions such as playing a card when they do not have enough food.
- When gaining a food token, we will need to make sure that the amount of food they can take from the bird feeder will be properly determined by the number of cards they have within the forest biome. We also need to guarantee that if the player chooses to trade a card for another food token, they will be able to properly select a card and that the card will be discarded.
- When laying eggs, we will need to make sure that the number of eggs that can be laid is properly given, and that the player is not able to lay more eggs than a bird card can handle. We also need to guarantee that if the player chooses to trade a card for an egg, they will be able to properly select a card and that the card will be discarded.
- When drawing cards, we need to make sure that the number of cards able to be drawn is properly given based on the number of birds placed down in the water biome. We will also need to guarantee that if the player chooses to trade an egg for another card, they will be given that choice and the result will be properly held out.

- When a card is drawn from the bird tray, we will need to check that a new card is drawn to replace the card drawn by the player.
- We will also need to verify that each card's value is accessible and connects without error to the main game. This can be done by using print statements for the keywords of the card within the main game, and making sure the values carry over.
- The abilities of each bird, if they have one, will need to be confirmed to be properly carried out when activated, played, or in between turns; whether it be a pink, brown, or white ability. We will check that everything is properly done through the use of print statements.
- The bird feeder will need to be confirmed to only refill when all dice are taken out of it, and rerolled only when all dice within the birdfeeder are of the same type.
- After each turn, we will need to verify that the number of actions cubes: 1. Decrease by 1 after a turn; 2. Increase the number of action cubes within the turn played by 1; 3. The last cube needs to be placed for scoring and in the right position. This can be done by using print statements.
- At the end of the round, we will need to verify the scoring of the round criteria as well as the transition from one round to the next with one less action cube. We will need to verify that only four rounds are run and that the points are properly displayed at the end of the game.
- Scoring will need to be verified by hand at the end of each round. If inaccurate, a debugging process will take place in which each step of the scoring process is outputted such that we can track down and eliminate the errors.
- The bonus cards must have their bonus points properly outputted. If any errors or inaccuracies are found, the bonusAction method of the Player class must be debugged.

More specific:

Game Class

- Verify that the game launches properly and that all variables, arrays, and objects receive their correct default values before entering into the game.
- Check that player turns go in proper rotation Player 1, Player 2, Player 3, Player 4 and then back to Player 1. Make certain it properly loops through all players and does not skip or repeat any one.
- Check that nextRound will reset all player action cubes, reinitialize the round variables, and add one to the round total. There must be a good transition from one round to the next.
- Verify that scores are calculated accurately through the sum of bird card points, egg points, bonus points, etc.
- Check that endGame is not called until after round four has been fully played out and calls for the final score as well as displays the score.
- Check that drawBird and drawBonus operate correctly in that one card is removed from the deck and added to the proper player's hand, reducing the deck total by one and preventing any duplicate draws. Ensure that the game can safely reshuffle decks without errors.
- Ensure that all randomization and distribution of cards behave fairly and have a consistent logic through multiple play-throughs.

Player Class

- Ensure that the player can draw food successfully and decrease the total food supply accordingly. Ensure that negative food values and impossible food values cannot occur.
- Ensure that playCard can only be executed when the player has enough food and there is space in the proper habitat. The game should offer a clear message not allowing the player to play if there are not enough necessary resources.
- Ensure that when a bird is played that it may only be placed in habitats listed in the (bitmask) habitat value. For example, a bird which lives in forests only may not be placed in grassland or wetlands.
- Ensure that the function layEggs increases each of the proper BirdCards egg count but never goes over the cap of eggs allowed for each bird.
- Ensure that effects from bird powers such as "on play" or "when activated" occur correctly and once as they should.

- Make sure that there is one action cube removed after every valid action, that invalid actions do not use action cubes.
- Ensure that all food tokens and egg counts are updated in the display after each player's action and match the backend values of the data.
- Check that bonusAction works correctly on final scoring on end game bonuses and utilizes the BonusCard rules and that points are assigned as they should.
- Ensure that the player data remains without decreasing or increasing between rounds that it retains all collected cards, eggs, and food tokens.

Board and Spot Classes

- Make sure that the board for each player correctly creates a 3 × 6 grid of Spots upon starting.
- Make sure that when a BirdCard is placed, the corresponding Spot recognizes that it is occupied and prints out the appropriate bird image or name.
- Check that addEgg stops once the BirdCard's maximum is reached. If any attempts are made to add eggs when the maximum has been reached, nothing happens.
- Confirm that removeEgg and addFood instantly update the total number of eggs or food, and that those changes are reflected not only in the logic of the game, but that the player's screen also shows these changes.
- Make sure that a Spot which already contains a card cannot have another card placed on it. If an attempt is made to place a card in an occupied Spot, the appropriate message should be produced.
- Make sure that getEggCount returns correctly the number of eggs in total for the player's board, at all times, even after that number has been decreased or increased.
- Check that the Spot objects are correctly reset at the beginning of a new game, removing eggs and food as the game rules recommend.

Enums (BirdCard and BonusCard)

- Ensure that all BirdCards and BonusCards are created with valid data, such as names, habitats, food costs, and maximum eggs.
- Verify that BirdCard toString prints correct and legible information for each card, when called. Confirm that BonusCard effects appropriately add their sums during score calculation and displays correct totals in the end game summary.
- Check that BirdCard powers appropriately trigger in the only correct instances, for example, whenever a bird is played or when a row is invoked by the player.
- Verify that all values in the enums are accessible and is error-free.

Testing Graphics

- As soon as the program is run, we will need to make sure the graphics are displayed with the correct panel (MenuPanel)
- After clicking the “start” button, we will need to make sure that the game progresses into the number of players screen. This can be done just by manually testing.
- We will need to make sure that clicking a button for the number of players will properly carry over the correct number of players and will start the game.
- After the number of players is set up, selecting the cards and food tokens must be verified to properly save their decision of which cards and specific food tokens to keep. We will also need to guarantee that clicking a card will magnify the card, and clicking the button to select it or clicking the cancel button will add it to their hand or go back.
- When on the bonus card page, we will need to verify that clicking the bonus card will actually count it as clicking the card and magnifying to show the card. We will also need to check that after magnifying, clicking the button to select it or clicking the cancel button will add it to their hand or go back.
- After all of the setup, we will need to make sure that all functions the player can do properly run.
- We will need to make sure that after clicking a function - playing a card, gaining a food, laying an egg, or drawing a card - an action cube is put on the appropriate location, and that as the player puts more action tokens on a specific turn type, that the action cubes will not obstruct the view of anything.
- We will also need to make sure that clicking the arrows to move forward or backward to view the previous or next player will move to the correct player and not in a different order.
- If a player clicks a card/bonus card in order to view it in a magnified view, we will need to make sure that this has the option to go back and that it properly works, and that the card needs to be shown large enough to be seen properly.
- We will also need to make sure that on the bird feeder screen, clicking a dice would properly give the right food token.

- For drawing a card, we will need to verify the same expectations for magnifying the card and selecting the correct card.
- We will need to check that the mouse click method works without any errors, so clicking in blank parts of the panel does not cause any error in compile time.
- We will need to verify that each button can be accurately linked to its designated action. A trigger system is not currently planned for, but it may be developed later on if necessary and simplifies management.
    - The trigger system, if developed, will be a simple structure of objects that contain coordinates and sizes. These objects will have methods to determine if a click is within bounds as well as an ID that connects it to a central method in the trigger system. This central method will run a switch statement on the ID to run its corresponding action.

MenuPanel

- Verify that the Play button changes correctly from menu to GamePanel without visual failure or program failure.
- Verify that the Exit button closes the window and program without errors or freezing, and no remained processes in the background.
- Check all mouse events (click, enter, press, release, exit) accurately respond to and result in correct actions or visual changes.
- Check all text and button labels are readable, centered, and properly aligned with all screen resolutions.
- Verify that resizing the game window doesn't result in distortion or overlap in any of the menu items.
- Check that on starting a game all player and game variables are initialized properly before entering the GamePanel.

GamePanel

- Check that the screen updates correctly after each player action, for example, drawing birds, laying eggs or gaining food.
- Verify that round number, turn order, cube number, number of eggs and number of food have all correctly updated from each move.
- Check that newly played birds appear in their correct grid space upon the player's board.
- Check that the changing of player correctly updates the board to that player's data, with an erasure of the previous player's visuals.
- Verify that all birds, food and egg icons remain clear and properly sized on all resolutions.
- Check that there are no errors or unexpected changes on screen from clicking areas outside of those in active zones.
- Verify that during the end of the round that the final scoreboard appears with all players' totals shown.
- Ensure that the animations, if included, correctly run and do not get out of synch with the logic of the game.
- Verify that there is no overlapping on screen after every action has taken place.

System Integration Testing

- Verify that the GUI buttons correctly invoke the logic functions, where appropriate, such as startGame, nextTurn, nextRound, calculateScore, error-free, and at the correct time

- Confirm that the visual and logical parts remain synchronized at all times, a cube spent in the backend of the program must instantaneously be put in the right spot upon the display screen.
- Verify that where the end of round/game takes place, the respective visuals are reset correctly but all player data and score totals are not lost.
- Check that the event listeners for mouse actions and button actions respectively are responsive and appropriately handle errors in invalid clicking.
- Verify that each player's board and data refresh with correctly when currently in the turning of players.
- Ensure that all transitions between MenuPanel and GamePanel occur without program failures and missing assets.
- Verify that the logic and graphics of objects are maintained and equal throughout play to avoid desynchronization.
- Ensure that the game can be run through four complete rounds with various players without freezing, crashing or desynchronized between GUI and logic layers.

Potential Conflicts

Just as any other project's creative process, we anticipate bumps on the road and know that it is unreasonable to expect the sheer amount of planning we have done to make this program the exception. Nevertheless, we hope to avoid being blindsided by an unexpected difficulty by predicting certain potential problems that may occur.

- Time constraints: Although we have done our best to plan for the weeks ahead and to try to minimize problems, we recognize that unforeseen events can occur and there may be days where we fall behind schedule despite doing everything we can. To minimize the impact of such days, we have agreed that in the event of an emergency, we will spend as much time needed during the weekends in order to catch up, and if this is not enough, the 7th week could be repurposed as make up days.
- Inefficient code: To mitigate this, we will have a process of transparency in which our fellow group members can peer review each other's code in order to ensure quality and brevity.
- Since we will be writing multiple classes at the same time, they may not be well connected. We will stick to the UML designed to correct this issue because it connects the classes. The members coding the classes will also point out any errors in the UML and we will work together to solve the problem by taking into account the viewpoints of all classes.
- In graphics, we may have a difficult drawing the JPanel based on different coordinates since we are all working on different classes at the same time. In order to overcome this issue, we will attempt to adhere to the graphical representation of the project shown in previous slides and estimate approximate points and test JPanel frequently as we go along. If we are to change the GUI, we will inform each other in advance.
- Additionally, if any programmer in our team faces any conflicts regarding their role, it is the other teammates' accountability to help because we are a group and the ignorance of problems will slow our progress.

General Timeline

Week number 1: October 19th - October 26th

Main goal: Create the structure of the project, startup all of the main classes, and establish the UML representation in the program.

Checklist:

- Make a structure of the whole project folder (essentially create packages for game, user interface cards, and player and store the respective classes in each package).
  - This will help with organization which is essential for planning.
- Give definitions to all of the enums, and the classes.
  - This includes the classes Wingspan, Game, Player, Board, Spot, MenuPanel, GamePanel and the enums BirdCard, and BonusCard.
- Write the constructors, private instance variables (encapsulation), and the placeholder methods.
- The UML diagram should show the respective relationships as follows.
  - Wingspan→ Game→ Player→ Board→ Spot→ BirdCard and BonusCard.
- Set up the Github shared folder for collaboration of each individual's work.

Simplified milestones that counts towards the whole project:

- All the classes are defined.
- UML diagrams are implemented in the program.
- The repository and folder are finalized.

Due date for this main goal: October 26th

*Tasks is synonymous with checklist

Week number 2: October 26th - November 2nd

Main goal: Focus on the integration between the Enums and the Player and Game class.

Checklist:

- Work on BirdCard enum by specifically programming the constants of each bird, with its attributes.
  - The attributes of the BirdCard include name, eggMax, habitats (bitmask), wingspan, foodCost, actionId, oneplay which will be the parameters of the constructor.
- Work on the BonusCard enum with its respective constants as well, although it will have a simpler structure.
  - The single attribute is the name which will be the parameter of the constructor.
- Work on the Game class in order to create the decks through the method in the Enum, which is Enum.values().
- Work on the functionality of the placeholder methods in the Player class that are storage related.
  - These methods include drawBird(), drawBonus(), discard(), addEgg()
- Make sure that the decks are successfully distributed to each player, and they are able to discard and draw other items in the game.

Simplified milestones that counts towards the whole project:

- Players can hold the cards(BirdCard and BonusCard)
- The initialization of the game successfully distributes the cards to the players.

Due date for the main goal: November 2nd

Week number 3: November 2nd - November 9th

Main goal: The main goal focus for this week will be to create a proficient turn system between the players in the game.

Checklist:

- Work on the methods in the Game class which correspond to the turn system.
  - startGame(), nextTurn(), nextRound(), endGame()
- Further expand the methods in the Player class for the flow of the game between the players.
  - playCard(), gainFood(), layEggs()
- Have the Board and Spot class connect to each other so the birds can be activated in the correct habitat.
- Track the state of the round during the game, and also have basic score calculation as well.
- Work on test cases for the players so the round rotation is successful.

Simplified milestones that counts towards the whole project:

- The systems of the turns amongst the players is working successfully.
- The rounds are adding on to the game and the scoring is working side by side as well.

Due date for this goal: November 9th

Week number 4: November 9th - November 16th

Main goal: Collaborative enhanced development of the graphical user interface(GUI).

Checklist:

- Form the layout for how the Menupanel will look like with the different buttons on the screen.
  - This is very important because this is essentially the aspect of the program the user interacts with.
- Build the GamePanel that displayed each players' food, eggs, and board as well. (The board for each player will not be shown all at once because we will keep a button for the user to toggle between the board in order to maintain organization.
- Code the mouseListener methods (we will be using mouseClicked(()) so the interactions are working.
- Essentially, link the Panel with the backend aspect of the programs
  - startGame(), Wingspan.setPanel()

Simplified milestones that counts towards the whole project:

- The menu and the game panel are working.
- The Gui is able to show data and it is able to respond to input.

Due date for this main goal: November 16th

Week number 5: November 16th - November 23th

Main goal: Further integration of the systems, completely finalize the whole scoring system, and make sure of the smooth interconnection between the GUI and logic.

Checklist:

- Focus on the full merging of the actions in the GUI and the logic of the Game class.
- Work on the Bonus card scoring methods through the development/use of the Bonus Card enum.
- Create the final version of the calculateScore() method and the logic for nextRound() as well.
- Ensure that the turn system is actually seamless and the displays of the round are working in cases of any small bugs from the third week.

Simplified milestones that counts towards the whole project:

- Now there should be a complete playable loop and the scoring should not be basic, but it should be fully developed.
- The effects of the Bonus cards are correctly working and functional.
- GUI is fully in sync with the backend of the program.

Due date for main goal: November 23rd

Week number 6: November 23th - November 30th

Main goal: This week will be focused on testing the game, and further developing upon the performance of the game**.**

Checklist:

- Run the unit tests for the classes and enums in the program.
  - These classes include Game, Board, Player, Spot, and the enums BirdCard and BonusCard.
- Make sure of all the gameplay actions in the game and that they accurately follow the rules of the Wingspan rulebook, and focus on the potential edge cases involved in the program which might cause the program to crash.
- Test different possible cases that involve the Graphics;some of these include when the user clicks on the blank screen(error occurring due to the mouseclicked method).
- Keep track of the results of the tests that we are carrying out in our program.

Simplified milestones that counts towards the whole project:

- Make sure that the tests are able to pass successfully.
- Check if the game is without any runtime errors.
- Make the game ready for any additional changes and further/extra polishing.

Due date for the main goal: November 30th

Week number 7: November 30th - December 5th

This week will most likely be a continuation of the next week depending on the extent of the errors that we are encountering in our program

Main goal: The main goal for this week is to make final touches to the project, and ensure that it runs smoothly. Help other teammates that are encountering conflicts in their role specific goals.

Checklist:

- Conduct the final round of testing in the game.
  - Essentially our group will play the game because it is a four player game and our group consists of four members.
- Perfect the performance of the GUI and the user experience
- Potentially make the UI of the game more

Simplified milestones that counts towards the whole project:

- Finish all the main goals.
- Focus on any conflicts that are present in the code.

Due Date for main goal: December 5th

Division of Workload

Weeks	Carter Feng	Fedhil Al-Shammary	Ansh Jain	Jake Guo
1	Form the basic MenuPanel, and GamePanel placeholders. Also work on the initialization of the enums.	Establish the structure of Board and Spot class and create the placeholder methods, and variables and link them together(Board will have a List of Spot objects)	Build the Player class with the planned variables and the empty methods that go along with the player class.	Create the skeleton classes for Game, and Wingspan.
2	Create the setup for GUI and create the layout for the frame.	Establish the structure of Board and Spot class and create the placeholder methods, and variables and link them together(Board will have a List of Spot objects)	Add more functionality to the Player class methods in order to successfully accomplish the necessary tasks.	He will be implementing the constructor for the Game, creating the players in the game, and developing the method handOutCards()
3	Begin adapting upon the GUI of the game by focusing on the transitions in between the turns.	Focus on and potentially finish the Spot.activateCard() and Board.playCard() logic	The main role will be to debug the actions of the players and their interactions with the components of the game itself.	He will be working on the turning logic and the progressive state of the rounds of the game.

4	Lead the whole team with the creation of GUI with all the handling of the events.	Make sure that the coordinates of the grids are in match with the visual of the board on the panel.	Make sure that the data that is being shown on the panel matches with the logic of the backend.	He will focus on the seamless and smooth integration of the game methods with the panel of the game.
5	Clean up and refine the GUI alongside the transitions within the GUI.	Work on the debugging of the board as well as the systems for activations of the cards.	Integrate the functions of the Player and further test the functionality of the gameplay	Focus on the tweaking and refinement of the logic that is within the Game class
6	Further better and optimize the (GUI) and fix any crashes or errors in the panels if necessary.	He will be very focused on testing the logic for the board logic.	Further expand on the potential test cases for the classes and the enums to conduct in the code, which are not covered in the section: Testing Data.	He will be the main leader of testing and fixing of the logic issues in the program.
7	Further better and optimize the (GUI) and fix any crashes or errors in the panels if necessary.	He will be very focused on testing the logic for the board logic.	Further expand on the potential test cases for the classes and the enums to conduct in the code, which are not covered in the section: Testing Data.	He will be the main leader of testing and fixing of the logic issues in the program.